

Programming Technology Documentation

3rd assignment task 3

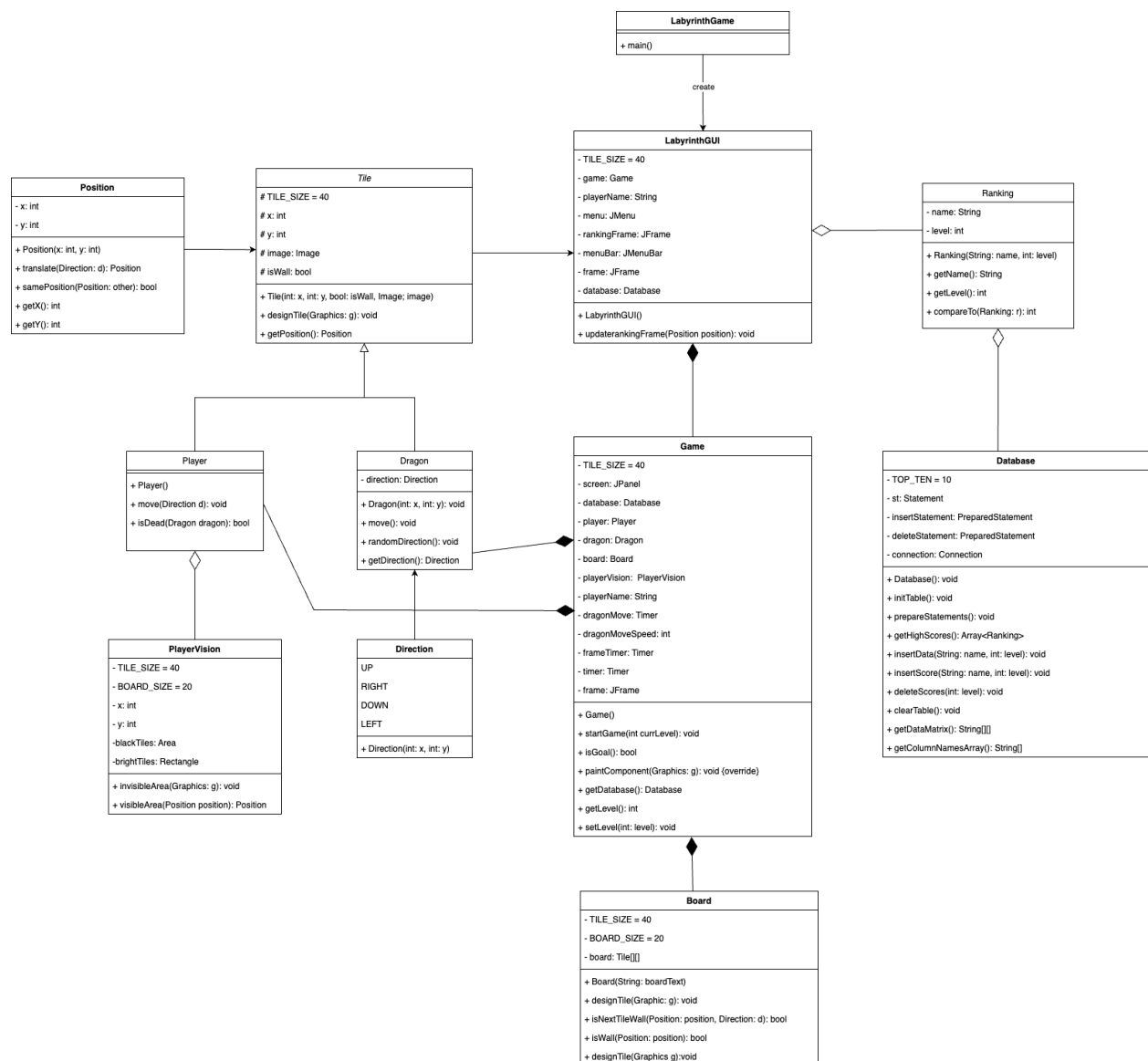
Jo Okinaga E3C64U

Task

Labyrinth

Create the Labyrinth game, where objective of the player is to escape from this labyrinth. The player starts at the bottom left corner of the labyrinth. He has to get to the top right corner of the labyrinth as fast he can, avoiding a meeting with the evil dragon. The player can move only in four directions: left, right, up or down. There are several escape paths in all labyrinths. The dragon starts off from a randomly chosen position, and moves randomly in the labyrinth so that it choose a direction and goes in that direction until it reaches a wall. Then it chooses randomly a different direction. If the dragon gets to a neighboring field of the player, then the player dies. Because it is dark in the labyrinth, the player can see only the neighboring fields at a distance of 3 units. Record the number of how many labyrinths did the player solve, and if he loses his life, then save this number together with his name into the database. Create a menu item, which displays a highscore table of the players for the 10 best scores. Also, create a menu item which restarts the game. Take care that the player and the dragon cannot start off on walls.

Class diagram



Method Descriptions

[Tile.java]

designTile(Graphics g): Put the image to the tile.

[Position.java]

translate (Direction d): get the next position based on direction

samePosition(Position other): check if the same position or not

[PlayerVision.java]

invisibleArea(Graphics g): the area where the player cannot see

visibleArea (Position position): the area where the player can see

[Player.java]

isDead(Dragon dragon): check if the player is dead or not

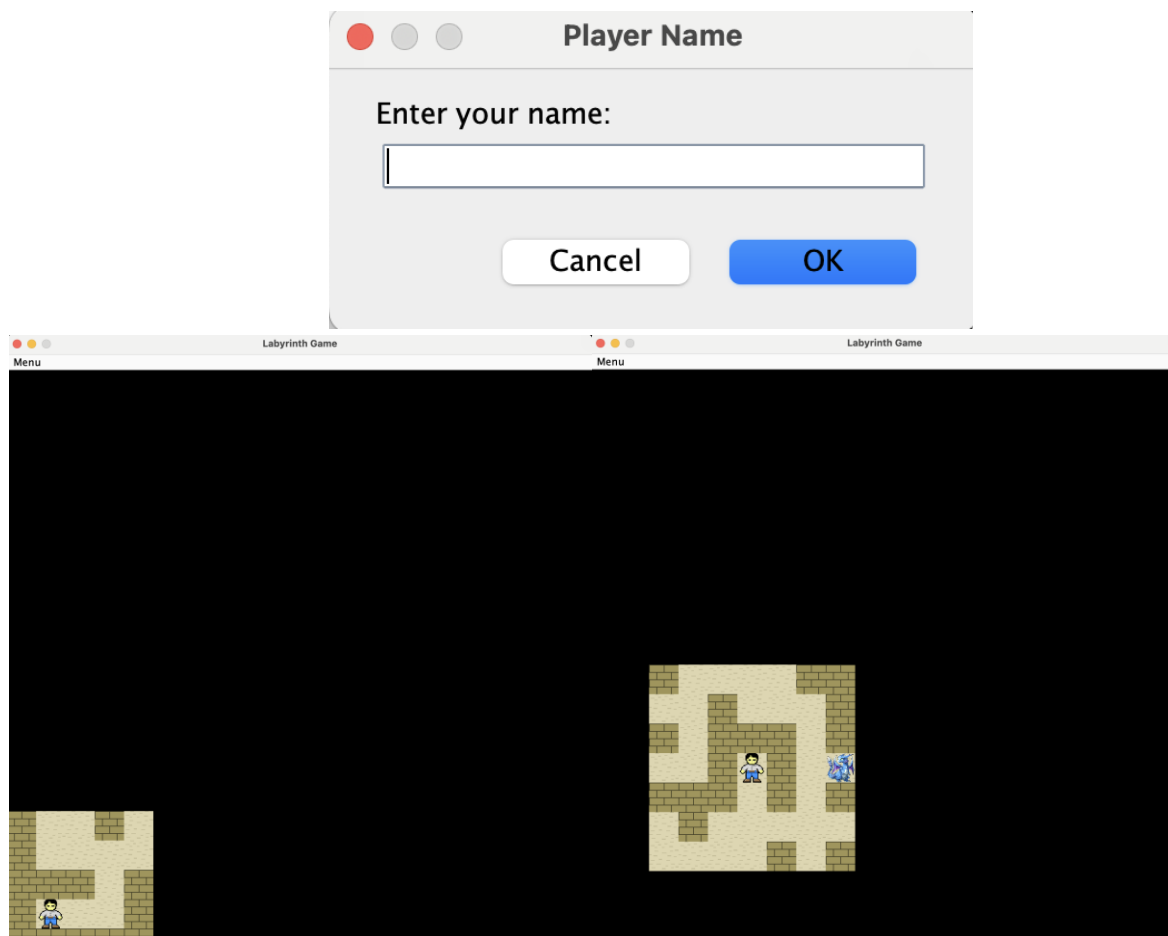
[Dragon.java]

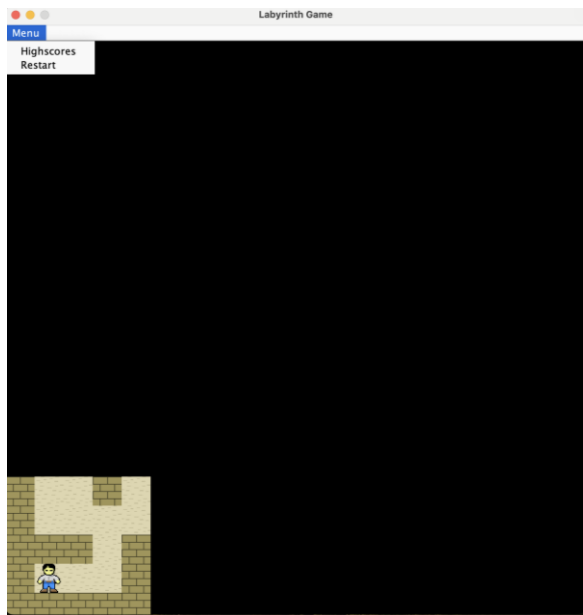
randomDirection (): decide dragon's next direction

[Game.java]

isGoal(): check if the player complete the board or not

Application Screenshots





Test Cases

Test Case 1: Display the main menu

- **AS A** user
- **I WANT TO** see the main menu options
- **GIVEN** the application is launched
- **WHEN** the main window is displayed
- **THEN** I should see buttons for "Start Game", "Settings", and "Exit".

Test Case 2: Start a new game

- **AS A** user
- **I WANT TO** start a new game
- **GIVEN** the main menu is visible
- **WHEN** I click the "Start Game" button
- **THEN** a new window with the labyrinth grid should appear.

Test Case 3: Ensure player starts at the initial position

- **AS A** user
- **I WANT TO** start at the initial position of the labyrinth
- **GIVEN** a new game has started
- **WHEN** I view the game board
- **THEN** the player should be at the starting point (e.g., top-left corner).

Test Case 4: Move the player

- **AS A** user
- **I WANT TO** move the player to an adjacent tile
- **GIVEN** a game is active
- **WHEN** I press an arrow key (up, down, left, right)
- **THEN** the player should move in the specified direction if the path is valid.

Test Case 5: Prevent movement into walls

- **AS A** user
- **I WANT TO** avoid moving into a wall
- **GIVEN** a game is active
- **WHEN** I try to move the player into a wall
- **THEN** the player's position should remain unchanged.

Test Case 6: Collect a treasure

- **AS A** user
- **I WANT TO** collect a treasure on the labyrinth grid
- **GIVEN** the player is on a tile with a treasure
- **WHEN** I move the player onto that tile
- **THEN** the treasure should disappear, and my score should increase by 1.

Test Case 7: Exit the labyrinth

- **AS A** user
- **I WANT TO** exit the labyrinth successfully
- **GIVEN** the player reaches the exit tile
- **WHEN** I move onto the exit tile
- **THEN** the game should display a success message.

Test Case 8: Prevent movement outside the grid

- **AS A** user
- **I WANT TO** avoid moving outside the grid boundaries
- **GIVEN** the player is at the edge of the grid
- **WHEN** I attempt to move beyond the grid limits
- **THEN** the player's position should remain unchanged.

Test Case 9: Detect game over condition

- **AS A** user
- **I WANT TO** know when the game ends
- **GIVEN** the player reaches the exit or loses all lives
- **WHEN** the game over condition is met
- **THEN** the game should display a game-over message and stop further actions.

Test Case 10: Restart game from main menu

- **AS A** user
- **I WANT TO** restart the game
- **GIVEN** the game has ended
- **WHEN** I click the "Restart" button in the main menu
- **THEN** the game should reset to its initial state.

Test Case 11: Exit game confirmation

- **AS A** user
- **I WANT TO** confirm before exiting the game
- **GIVEN** the main menu is visible
- **WHEN** I click the "Exit" button
- **THEN** a confirmation dialog should appear, and the application should close only if I select "Yes".

Test Case 12: Pause the game

- **AS A** user
- **I WANT TO** pause the game
- **GIVEN** the game is active
- **WHEN** I press the "Pause" button
- **THEN** the game should freeze, and a "Game Paused" message should appear.

Test Case 13: Resume the game

- **AS A** user
- **I WANT TO** resume the paused game
- **GIVEN** the game is paused
- **WHEN** I press the "Resume" button
- **THEN** the game should continue from where it was paused.

Test Case 14: Display remaining lives

- **AS A** user
- **I WANT TO** see how many lives I have left
- **GIVEN** the game is active
- **WHEN** I look at the game HUD
- **THEN** the remaining lives should be displayed.

Test Case 15: Lose a life when hitting a trap

- **AS A** user
- **I WANT TO** lose a life when stepping on a trap
- **GIVEN** the player moves onto a trap tile
- **WHEN** I land on the trap
- **THEN** my remaining lives should decrease by 1.

Test Case 16: Game over when no lives left

- **AS A** user
- **I WANT TO** end the game when I lose all lives
- **GIVEN** the player's lives reach 0
- **WHEN** I attempt another move
- **THEN** the game should display "Game Over" and stop further actions.